

4 时序逻辑电路建模

4.1 锁存器

4.2 时序电路建模基础

4.3 触发器

4.4 寄存器和移位寄存器

4.5 同步计数器

4.6 Verilog HDL函数与任务的使用

4.7 m序列码产生电路设计



山东大学
SHANDONG UNIVERSITY

信息科学与工程学院
School of Information Science and Engineering(ISE)

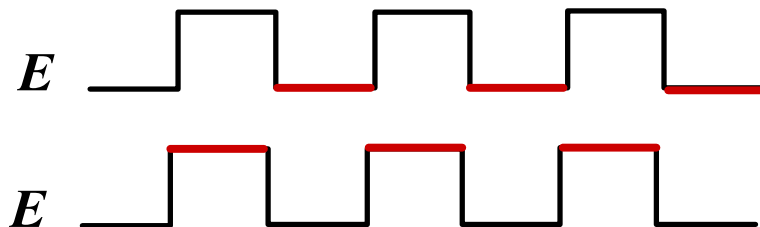
锁存器和触发器的基本特性

- 锁存器和触发器是构成时序逻辑电路的基本逻辑单元，它们具有存储数据的功能。
- 每个锁存器或触发器都能存储1位二值信息，所以又称为存储单元或记忆单元。
- 若输入信号不发生变化，锁存器和触发器必然处于其中一种状态，且一旦状态被确定，就能自行保持不变，即长期存储1位二进制数。
- 电路在输入信号的作用下，会从一种稳定状态转换成为另一种稳定状态。

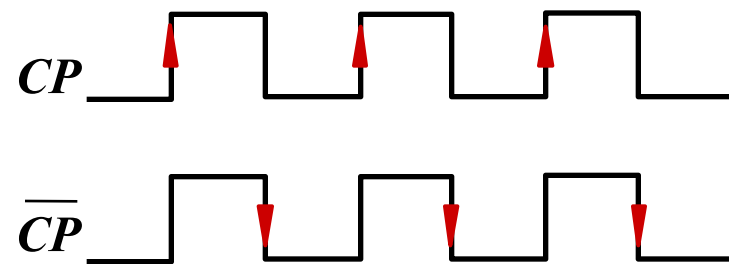


锁存器与触发器的区别

锁存器——没有时钟输入端，对脉冲电平敏感的存储电路，在特定输入脉冲电平作用下改变状态。



触发器——每一个触发器有一个时钟输入端。对脉冲边沿敏感的存储电路，在时钟脉冲的上升沿或下降沿的变化瞬间改变状态。



4.1 锁存器 (Latch)

4.1.1 基本SR锁存器

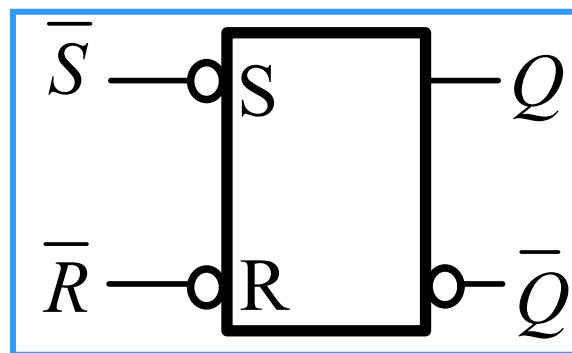
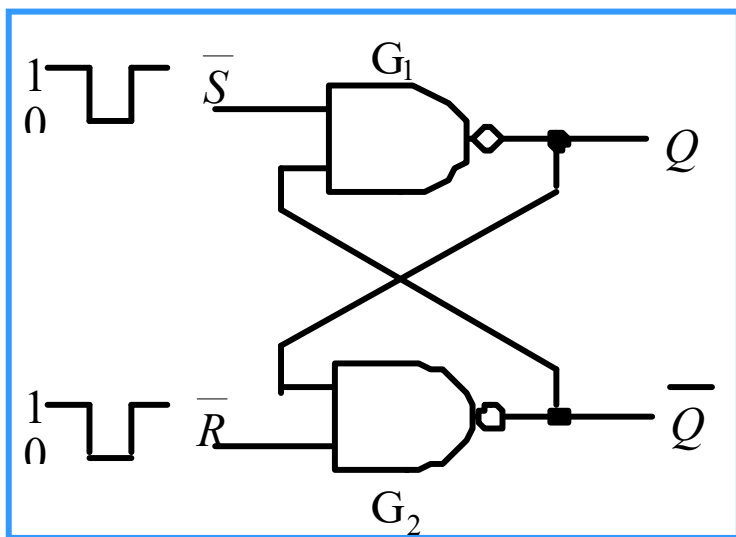
4.1.2 门控D锁存器

4.1.3 门控D锁存器的Verilog HDL建模



4.1.1 基本SR锁存器

1. 用与非门构成的基本SR锁存器

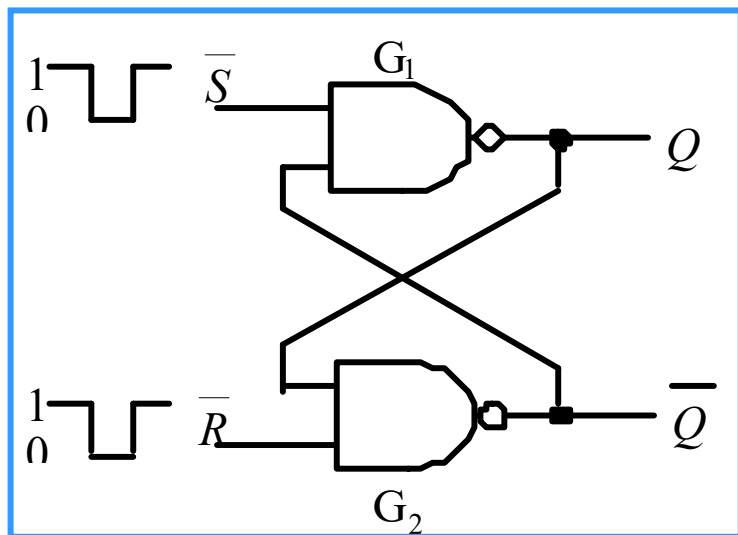


国标逻辑符号

方框外侧输入端的小圆圈和信号名称上面的小横线均表示输入信号是低电平有效的，同时为了区别，这种锁存器有时也称为基本 $\bar{S}\bar{R}$ 锁存器。

4.1.1 基本SR锁存器

1. 用与非门构成的基本SR锁存器

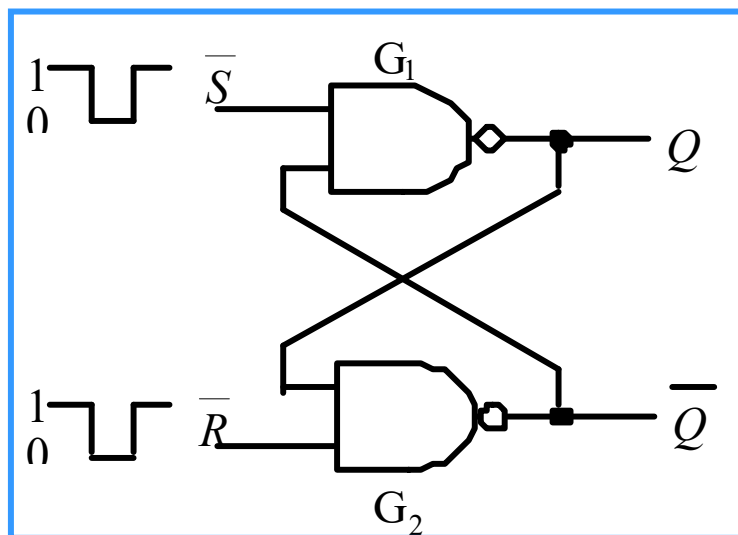


现态： $\overline{R}$ 、 $\overline{S}$ 信号作用前 Q 端的状态，
现态用 Q^n 表示。

次态： $\overline{R}$ 、 $\overline{S}$ 信号作用后 Q 端的状态，
次态用 Q^{n+1} 表示。

4.1.1 基本SR锁存器

a. 电路图



b. 功能表

$\overline{R}$	$\overline{S}$	Q	$\overline{Q}$
1	1	不变	不变
1	0	1	0
0	1	0	1
0	0	1	1

约束条件:

$$\overline{S} + \overline{R} = 1$$

例 当 $\bar{S}$ 、 $\bar{R}$ 的波形如下图虚线上边所示，试画出 Q 和 $\bar{Q}$ 对应的波形（假设原始状态 $Q=0$ ）。

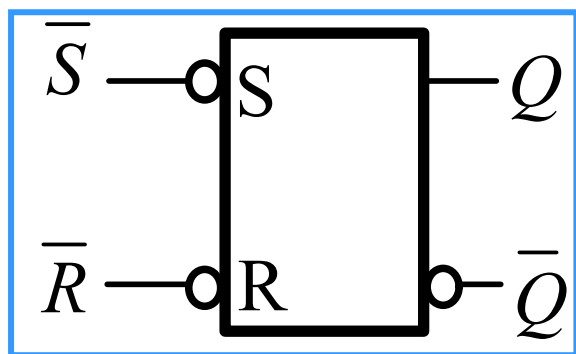
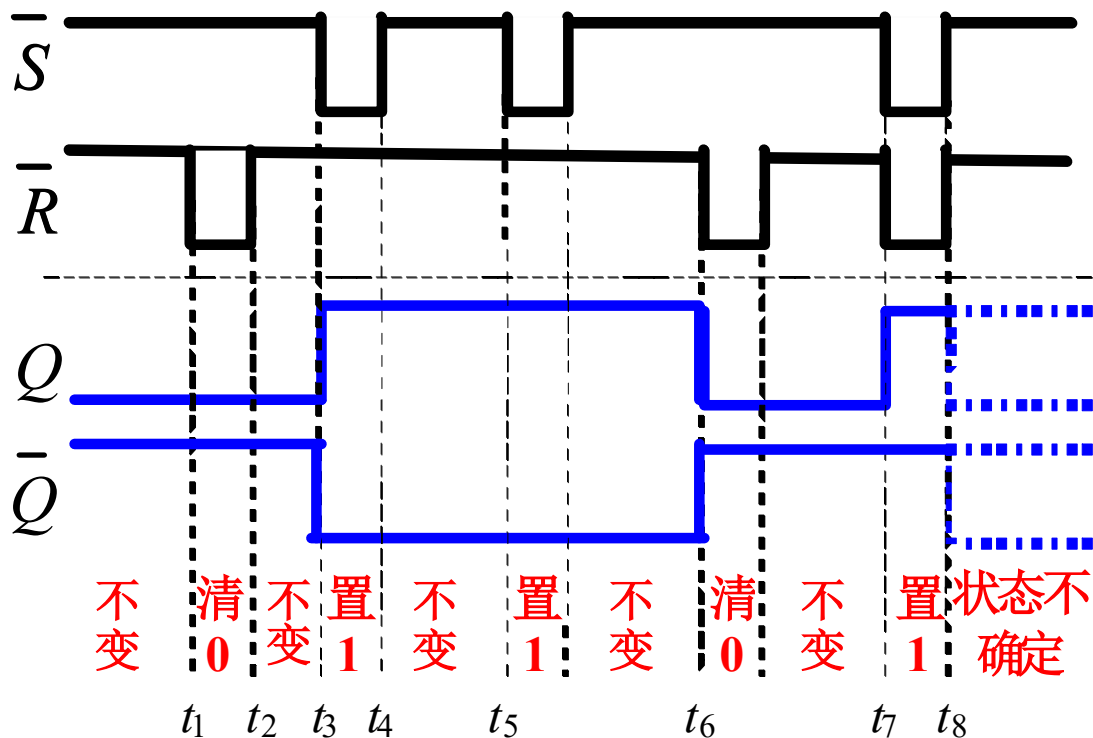


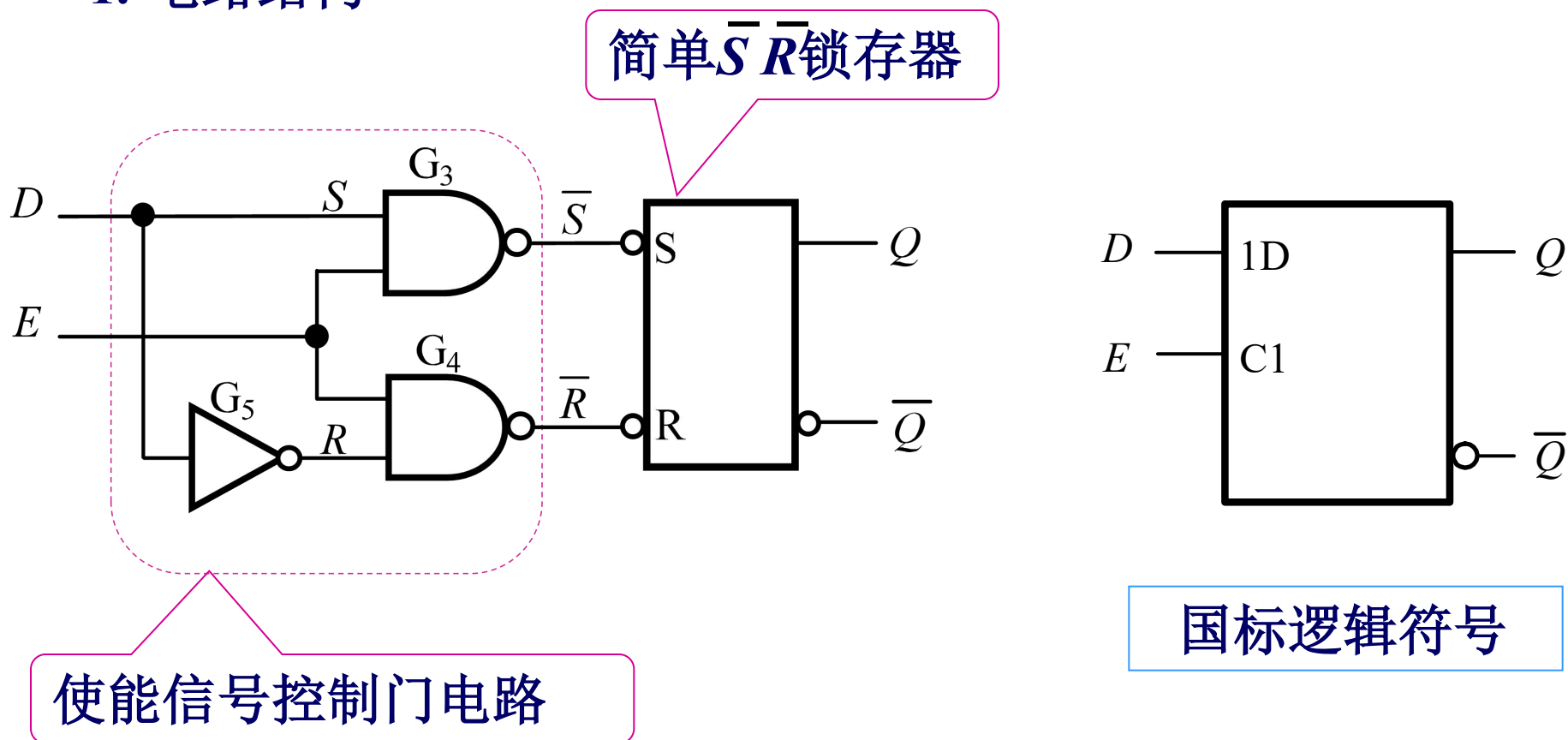
图4.1.1 (b)

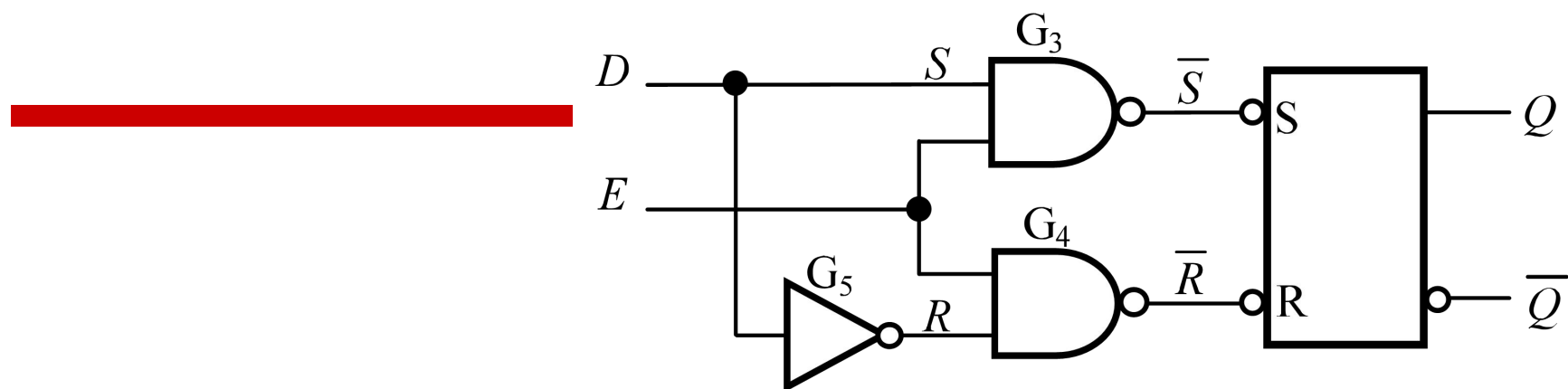
$\bar{R}$	$\bar{S}$	Q	$\bar{Q}$
1	1	不变	不变
1	0	1	0
0	1	0	1
0	0	1	1



4.1.2 门控D锁存器

1. 电路结构





- 当 $E = 0$ 时， $\bar{S} = \bar{R} = 1$ ，无论 D 取什么值， Q 保持不变。
- 当 $E = 1$ 时，
 - $D=1$ 时， $\bar{S}=0$ ， $\bar{R}=1$ ， Q 被置1；
 - $D=0$ 时， $\bar{S}=1$ ， $\bar{R}=0$ ， Q 被置0。
- 在 $E=1$ 期间， D 值将被传输到输出端 Q ，而当 E 由1跳变为0时，锁存器将保持跳变之前瞬间 D 的值。因此，D锁存器常被称为透明锁存器（Transparent Latch）。

特性表和特性方程

表 4.2.4 · D 锁存器的特性表

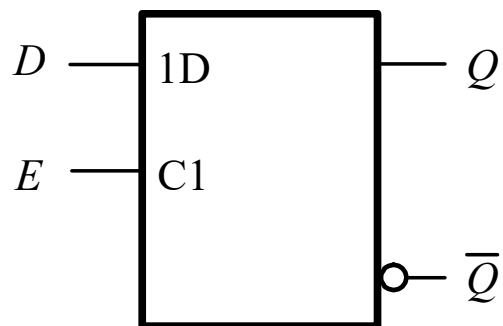
E	D	Q^n	Q^{n+1}	功 能
0	×	0	0	保持
0	×	1	1	
1	0	0	0	置 0
1	0	1	0	
1	1	0	1	置 1
1	1	1	1	

Q^{n+1}	DQ^n	00	01	11	10
E	0	0	1	1	0
	1	0	0	1	1

图 4.2.8 卡诺图

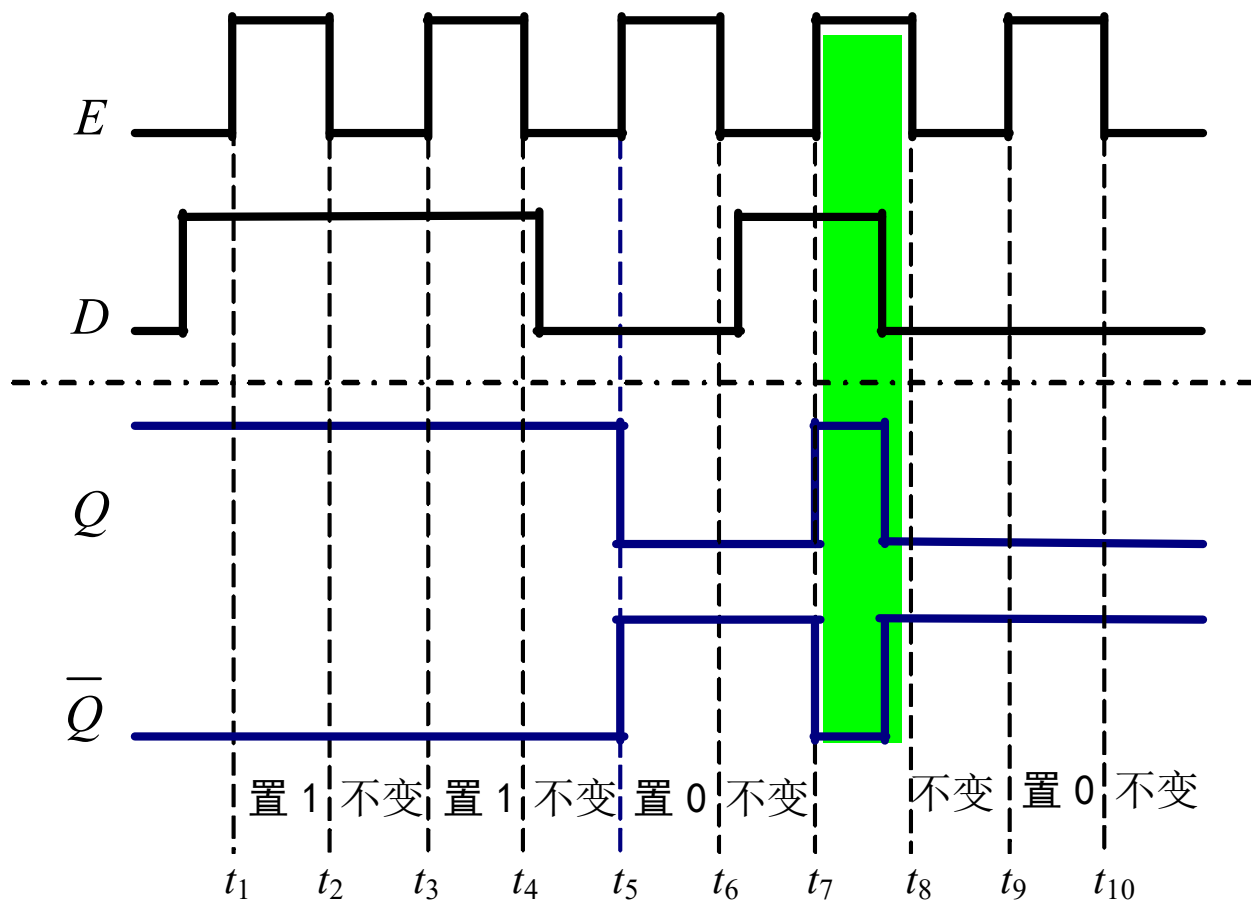
$$Q^{n+1} = \overline{E} \cdot Q + E \cdot D$$

波形图



初始状态
为 $Q=1$

$E \backslash DQ^n$	Q^{n+1}			
	00	01	11	10
0	0	1	1	0
1	0	0	1	1



4.1.3 门控D锁存器的Verilog HDL建模

1. 试对图4.1.3所示的D锁存器进行建模。

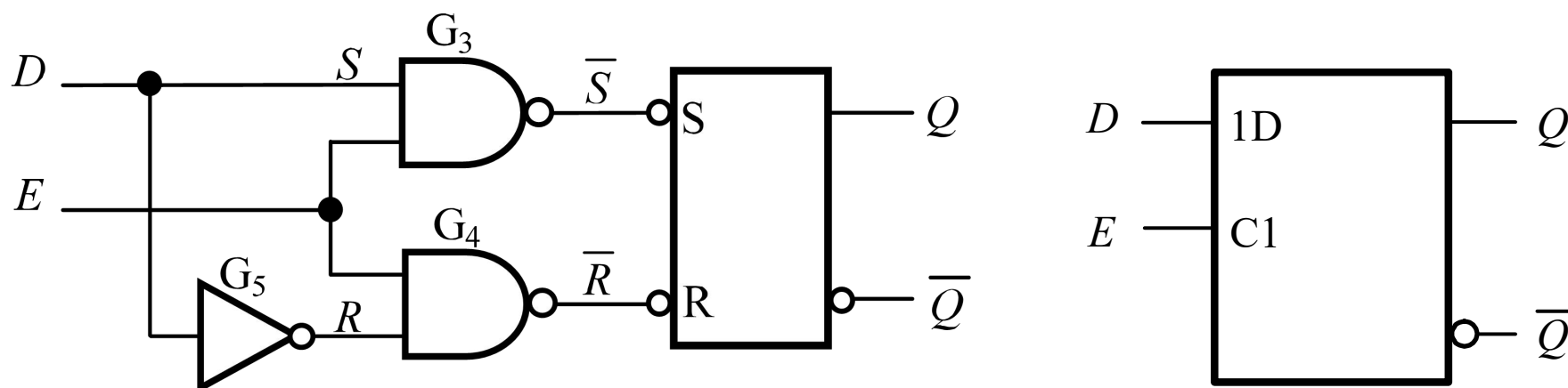


图4.1.3 D锁存器

```
//版本1: Structural description of a D latch
module Dlatch_Structural (E, D, Q, Q_);
    input E, D;
    output Q, Q_;
    wire R_, S_;
        nand N1 (S_, D, E);
        nand N2 (R_, ~D, E);
        SRlatch_1 N3 (S_, R_, Q, Q_);
endmodule
```

```
//Structural description of a SR-latch
module SRlatch_1 (S_, R_, Q, Q_);
    input S_, R_;
    output Q, Q_;
    nand N1 (Q, S_, Q_);
    nand N2 (Q_, R_, Q);
endmodule
```

版本1的特点:

- 第一个版本根据图4.1.3使用基本的逻辑门元件，采用结构描述风格，编写了两个模块，这两个模块可以放在一个文件中，文件名为**Dlatch_Structural.v**。
- 在一个文件中可以写多个模块，其中有一个是主模块（或者称为顶层模块）。
- 文件名必须使用顶层模块名。本例中**Dlatch_Structural**是主模块，它调用**SRlatch_1**模块。



```
//版本2: Behavioral description of a D latch
module Dlatch_bh (E, D, Q, Q_);
    input E, D;
    output Q, Q_;
    reg Q;
    assign Q_ = ~Q;
    always @ (E or D)
        if (E)
            Q <= D;    //当使能有效时, 输出跟随输入变化
        else
            Q <= Q;    //保持不变
endmodule
```



版本2的特点:

- 第二个版本采用功能描述风格的代码，不涉及到实现电路的具体结构，靠“算法”实现电路操作。对于不太喜欢低层次硬件逻辑图的人来说，功能描述风格的Verilog HDL是一种最佳选择。其中“<=”为非阻塞赋值符。
- 注意：
 - **always**内部不能使用**assign**。
 - 在写可综合的代码时，建议明确地定义**if—else**中所有可能的条件分支，否则，就会在电路的输出部分增加一个电平敏感型锁存器。



4.2 时序电路建模基础

Verilog行为级描述用关键词initial或always，但initial是面向仿真，不能用于逻辑综合。always是无限循环语句，其用法为：

always@ (事件控制表达式（或敏感事件表））

begin

块内局部变量的定义；

过程赋值语句；

end



4.2.1 阻塞型赋值语句与非阻塞型赋值语句

- 在always语句内部的过程赋值语句有两种类型：
 - 阻塞型赋值语句 (Blocking Assignment Statement)
 - 非阻塞型赋值语句 (Non-Blocking Assignment Statement)
- 使用的运算符如下：
 - 赋值算符(=)：阻塞型过程赋值算符
前一条语句没有完成赋值过程之前，后面的语句不可能被执行。
 - 赋值算符(<=)：非阻塞型过程赋值算符
一条非阻塞型赋值语句的执行，并不会影响块中其它语句的执行。



阻塞型过程赋值与非阻塞型过程赋值

//Blocking (=)

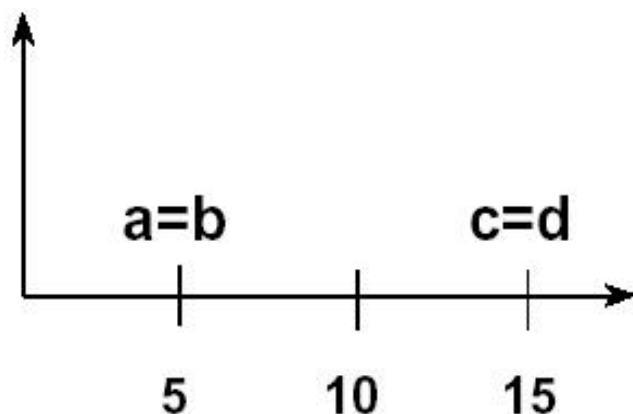
initial

begin

#5 a = b;

#10 c = d;

end



//Nonblocking (<=)

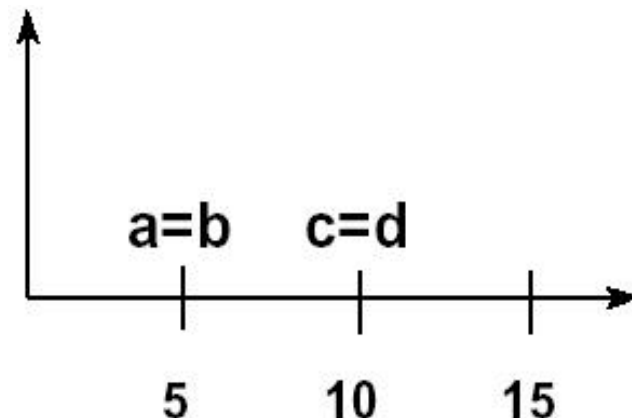
initial

begin

#5 a <= b;

#10 c <= d;

end



注意:

- 在可综合的电路设计中，一个语句块的内部**不允许同时**出现阻塞型赋值语句和非阻塞型赋值语句。
- 在时序电路的设计中，**建议采用非阻塞型赋值语句 $\leq$** 。



4.2.2 事件控制语句

- 用always语句描述硬件电路的逻辑功能时，在always语句中@符号之后紧跟着“事件控制表达式”。
- 逻辑电路中的敏感事件通常有两种类型：电平敏感事件和边沿触发事件。
- 在组合逻辑电路和锁存器中，输入信号电平的变化通常会导致输出信号变化，在Verilog HDL中，将这种输入信号的电平变化称为电平敏感事件。
- 在同步时序逻辑电路中，触发器状态的变化仅仅发生在时钟脉冲的上升沿或下降沿，Verilog HDL中用关键词posedge（上升沿）和negedge（下降沿）进行说明，这就是边沿触发事件。



敏感事件分为电平敏感事件和边沿触发事件：

电平敏感事件（如锁存器）：

always@ (sel or a or b)

always@ (sel,a,b)

sel、a、b中任意一个电平发生变化，后面的过程赋值语句将执行一次。

边沿敏感事件（如触发器）：

always@(posedge CP or negedge CR)

CP的上升沿或CR的下降沿来到，后面的过程语句就会执行。



边沿触发事件

- 在**always**后面的边沿触发事件中，有一个事件必须是**时钟事件**，还可以有多个异步触发事件，多个触发事件之间用关键词 **or** 进行连接，例如，语句

always @(posedge CP or negedge Rd_ or negedge Sd_)

- 在**Verilog 2001**标准中，可以使用逗号来代替**or**。例如，

always @(posedge CP, negedge Rd_, negedge Sd_)

- **posedge CP** 是时钟事件， **negedge Rd_**和**negedge Sd_**是异步触发事件。**如果没有时钟事件，只有异步事件，就会出现语法错误。**



4.3 D触发器

4.3.1 D触发器的逻辑功能

4.3.2 有清零输入和预置输入的D触发器

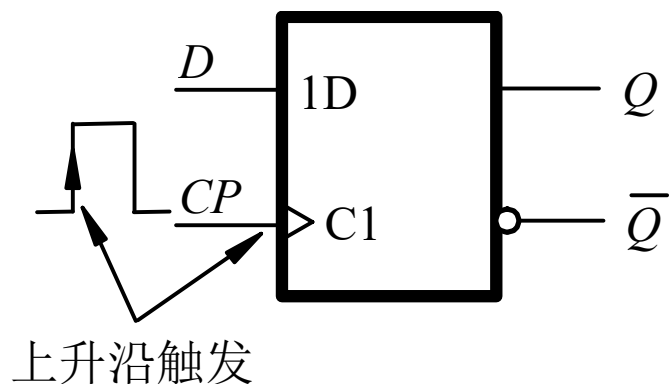
4.3.3 有使能端的D触发器

4.3.4 D触发器及其应用电路的Verilog HDL
建模

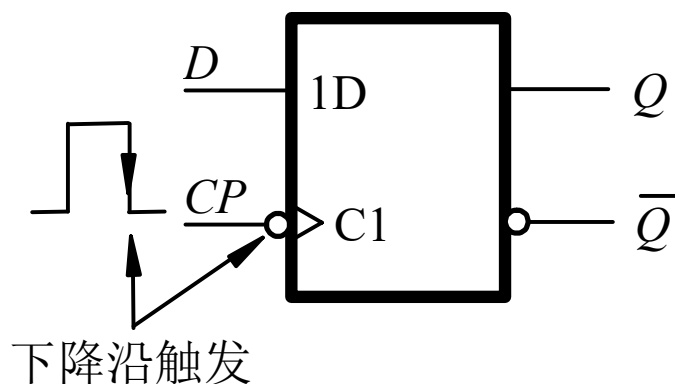


4.3.1 D触发器的逻辑功能

1. D触发器的逻辑符号



(a) 上升沿触发



(b) 下降沿触发

- 把 CP 有效沿到来之前电路的状态称为**现态**，用 Q^n 表示。
- 把 CP 有效沿到来之后，电路所进入的新状态称为**次态**，用 Q^{n+1} 表示。

4.3.1 D触发器的逻辑功能

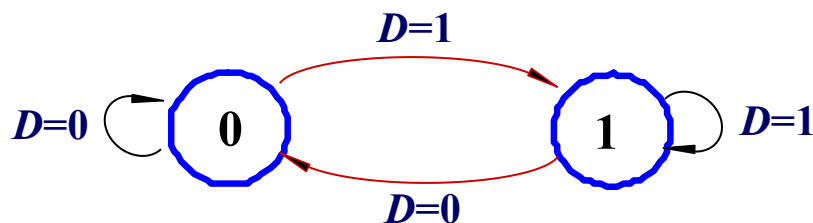
2. 特性表

D	Q^n	Q^{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

3. 特性方程

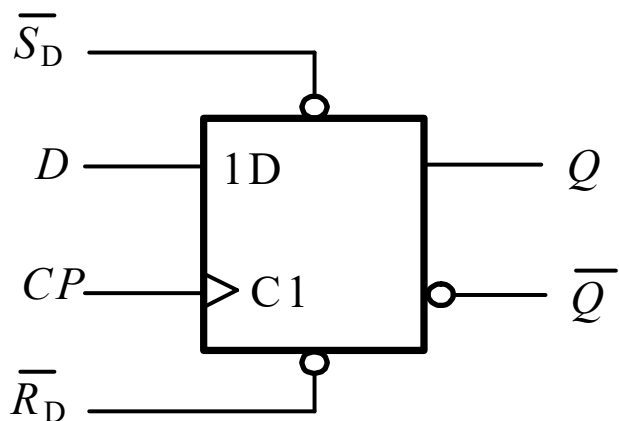
$$Q^{n+1} = D$$

4. 状态图

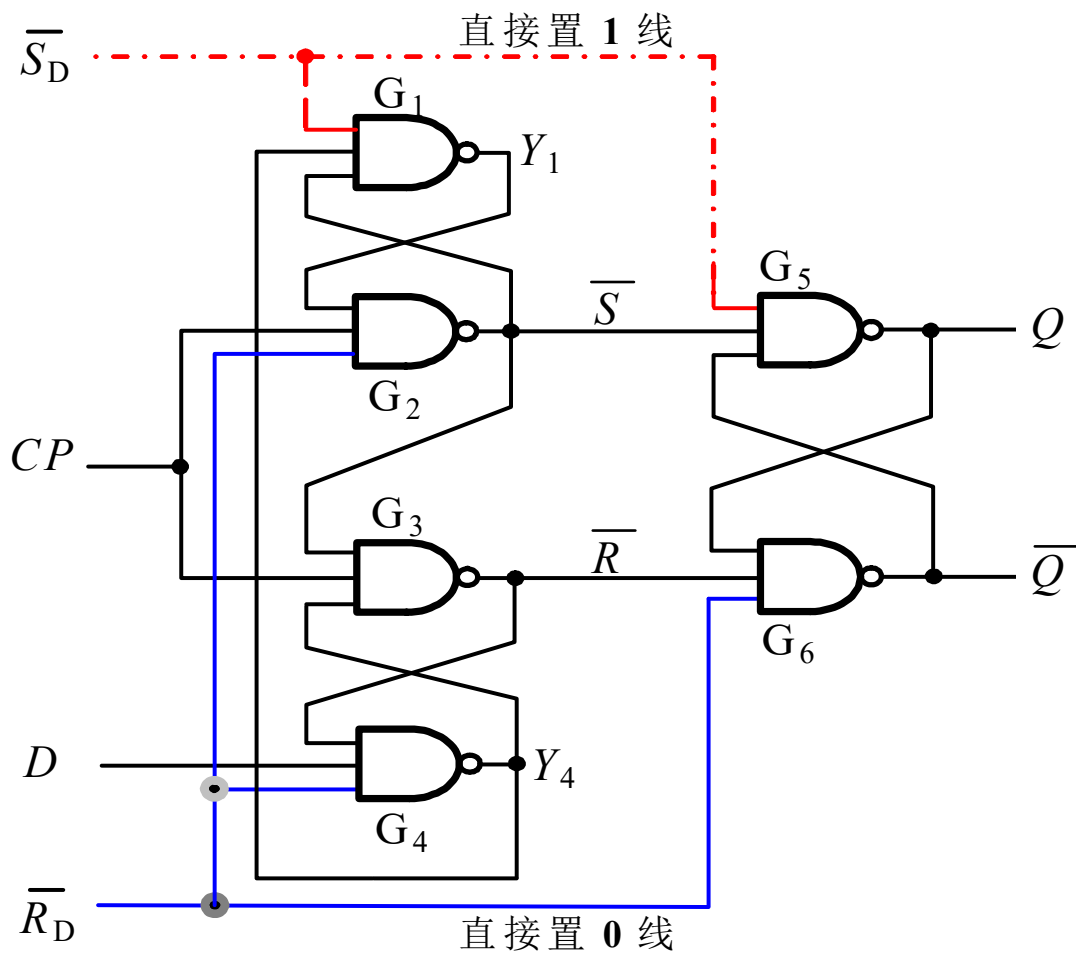


4.3.2 有清零输入和预置输入的D 触发器

由于直接置1和清零时跟 CP 信号无关，所以称置1、清零操作是**异步置1**和**异步清零**。



(a)



(b)

4.3.2 有清零输入和预置输入的D 触发器

直接置1和直接清零的过程如下：

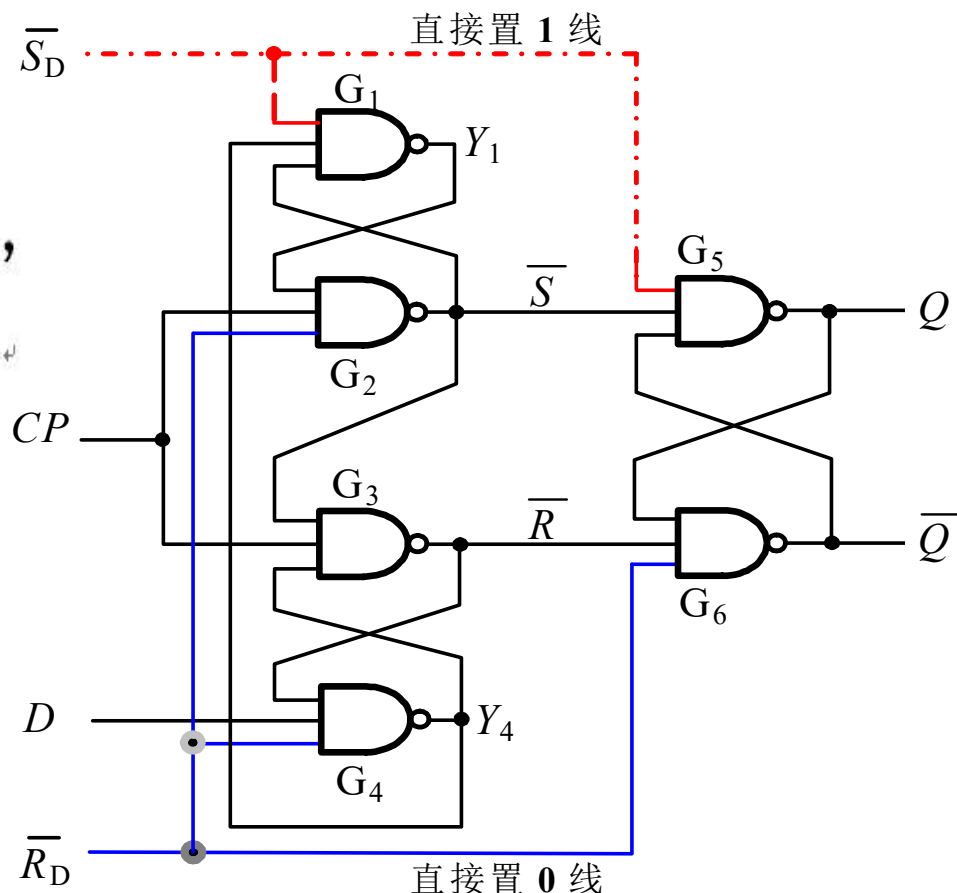
(1) 当 $\bar{S}_D = 0$, $\bar{R}_D = 1$ 时, 使得 $Y_1 = 1$,

$$\bar{S} = Y_1 \cdot CP \cdot \bar{R}_D = \bar{CP}, \quad \bar{R} = \bar{S} \cdot CP \cdot Y_4 = 1,$$

于是 $Q = 1$, $\bar{Q} = 0$, 即将输出 Q 直接置 1。

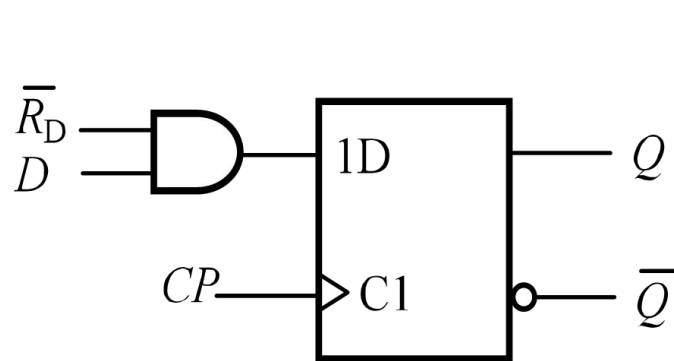
(2) 当 $\bar{S}_D = 1$, $\bar{R}_D = 0$ 时,

使得 $\bar{S} = 1$, 于是 $Q = 0$, $\bar{Q} = 1$, 即将输出 Q 直接清零。

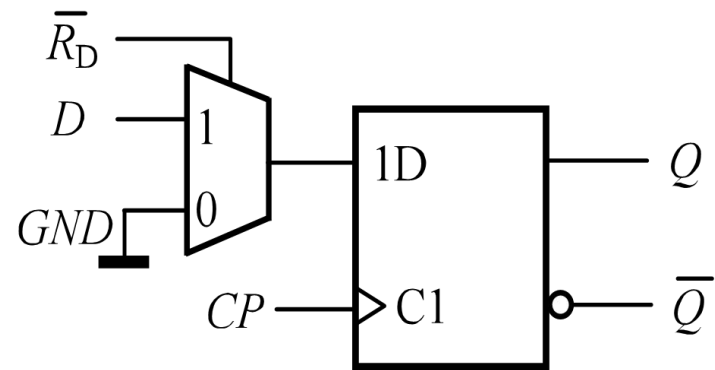


有同步清零端的 D 触发器

- 所谓**同步清零**是指在清零输入信号有效，并且 CP 的有效边沿到来时，才能将触发器清零。



(a)



(b)

图4.3.4 有同步清零输入端的D触发器

(a) 实现同步清零的方案之一

(b) 实现同步清零的方案之二

4.3.3 有使能端的D触发器

□ 功能:

- $En=0$, Q 保持不变。
- $En=1$, 在 CP 作用下,
 $Q = D$ 。

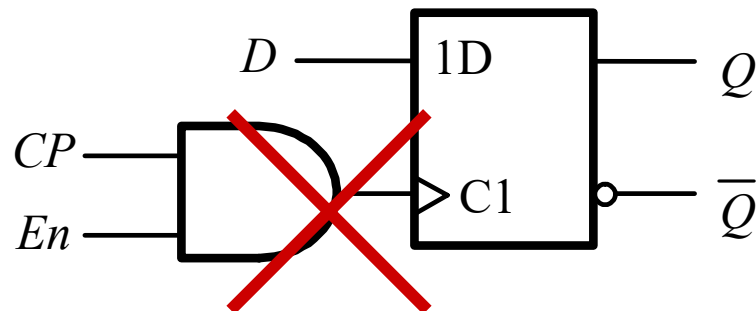
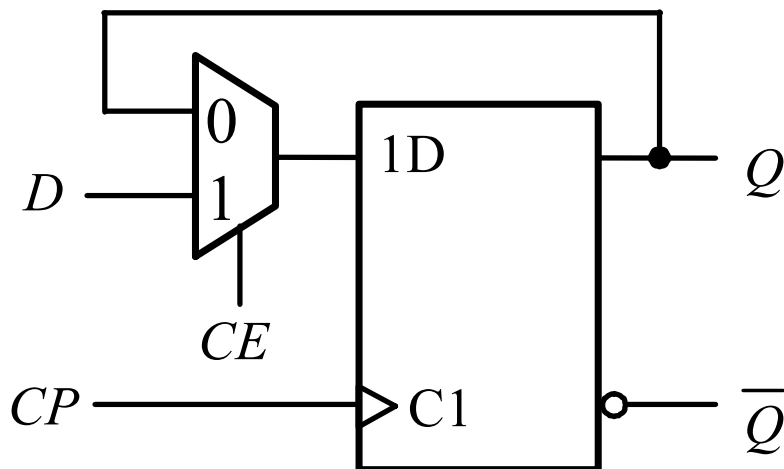
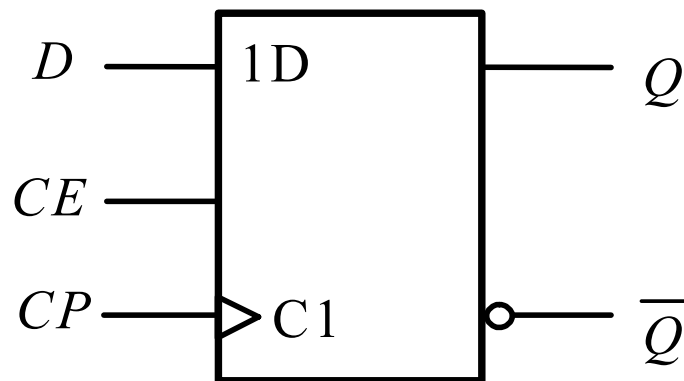


图 4.3.5 用门控制时钟的电路



$$Q^{n+1} = \overline{CE} \cdot Q^n + CE \cdot D$$



逻辑符号

4.3.4 D触发器及其应用电路的Verilog HDL建模

例1. 试对图4.3.3所示的带有异步清零和异步置位的边沿D触发器进行建模。

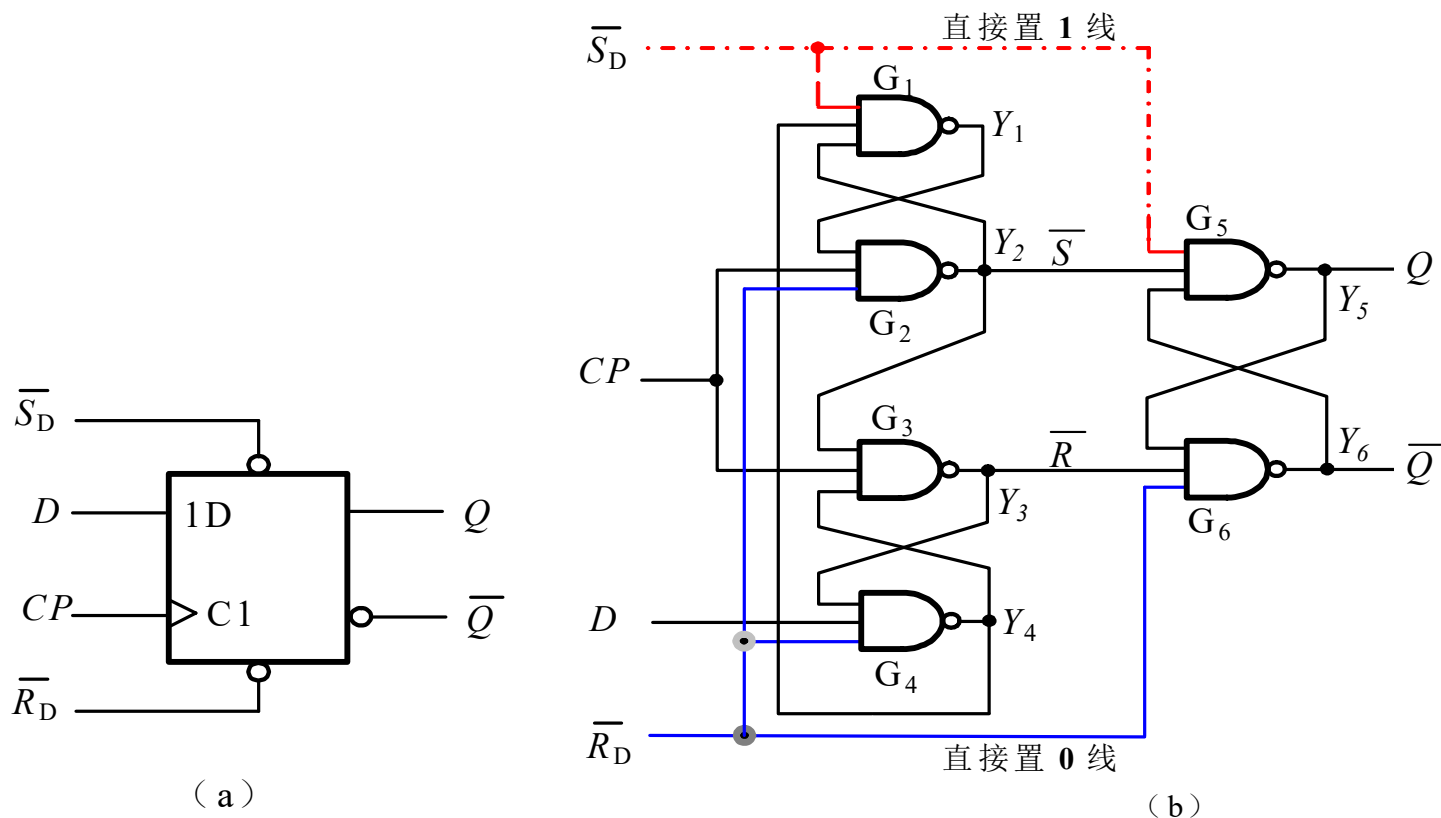


图4.3.3 有异步输入端的D触发器

//版本1:

```

module Set_Rst_DFF (Q,Q_,D,CP,Rd_,Sd_);
    output Q,Q_;
    input D,CP,Rd_,Sd_;
    wire Y1,Y2,Y3,Y4,Y5,Y6;

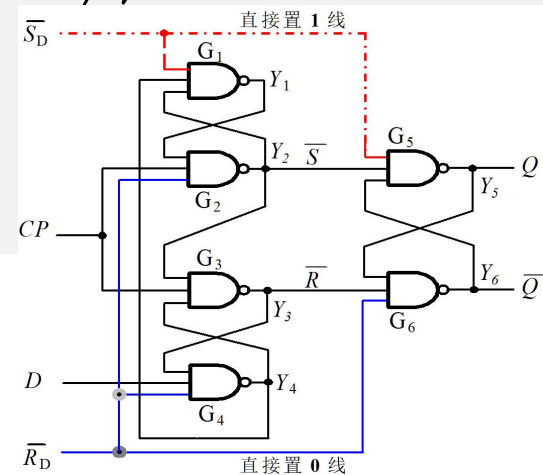
    assign #5 Y1 = ~(Sd_ & Y2 & Y4);
    assign #5 Y2 = ~(Rd_ & CP & Y1);
    assign #5 Y3 = ~(CP & Y2 & Y4);
    assign #5 Y4 = ~(Rd_ & Y3 & D);
    assign #5 Y5 = ~(Sd_ & Y2 & Y6);
    assign #5 Y6 = ~(Rd_ & Y3 & Y5);

    assign Q = Y5;
    assign Q_ = Y6;

endmodule

```

- 版本1根据该图使用连续赋值语句来建模，在**assign**语句中的**#5**表示给每个与非门加5个单位时间的传输延迟。



//版本2

```
module Set_Rst_DFF_bh (Q,Q_,D,CP,Rd_,Sd_);  
    output reg Q;  
    output Q_;  
    input D,CP,Rd_,Sd_;  
  
assign Q_ = ~Q;  
  
always @(posedge CP or negedge Sd_ or negedge Rd_)  
    if (~Sd_)           //等同于: if (Sd_ == 0)  
        Q <= 1'b1;  
    else if (~Rd_)  
        Q <= 1'b0;  
    else  
        Q <= D;  
endmodule
```

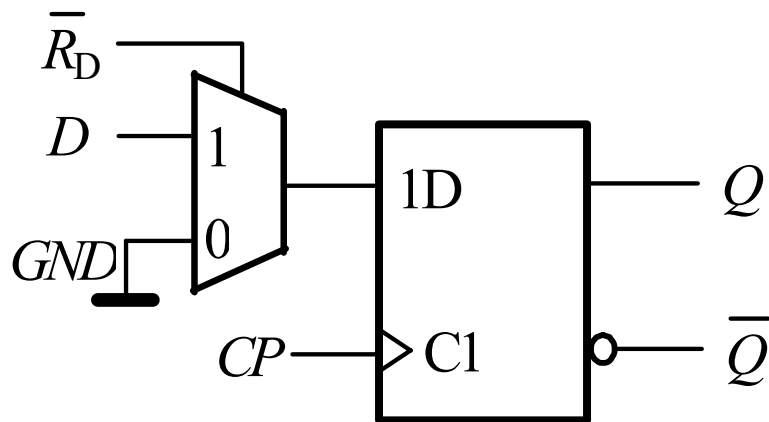
版本2的特点:

- 采用功能描述风格，使用always和if-else对输出变量赋值。
- negedge Sd_是一个异步事件，它与if ($\sim$ Sd_) 必须匹配，negedge Rd_是另一个异步事件，它与if ($\sim$ Rd_) 必须匹配。
 - 当Sd_为0时，将输出Q置1；
 - 当Sd_=1且Rd_=0时，将输出Q置0；
 - 当Sd_和Rd_均不为0，且时钟CP的上升沿到来时，将输入D传给输出Q。
- 注意，如果置1事件、置0事件和时钟事件同时发生，则置1事件的优先级别最高、置0事件的次之，时钟事件的优先级最低。



例2 具有同步清零功能的上升沿D触发器。

```
module Sync_rst_DFF (Q,D,CP,Rd_);  
    output reg Q;  
    input D, CP, Rd_;  
  
    always @(posedge CP)  
        if ( !Rd_ )  
            Q <= 0;  
        else  
            Q <= D;  
endmodule
```



例4 试用功能描述风格对图4.3.7所示电路进行建模，并给出仿真结果。

解：（1）设计块：使用always和if-else语句对输出变量赋值，其代码如下。

```
`timescale 1 ns/ 1 ns
module _2Divider (Q,CP,Rd_);
    output reg Q;
    input CP,Rd_;
    wire D;
    assign D = ~Q;
    always @(posedge CP or
            negedge Rd_)
        if(~Rd_)    Q <= 1'b0;
        else        Q <= D;
endmodule
```

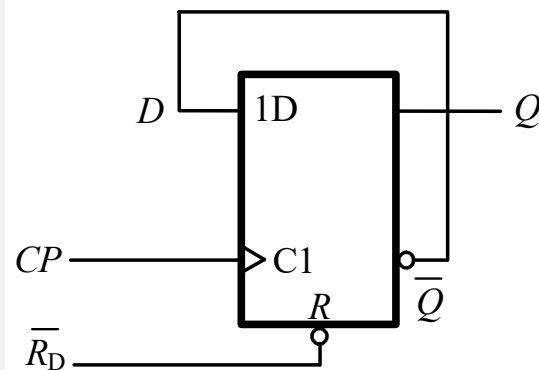
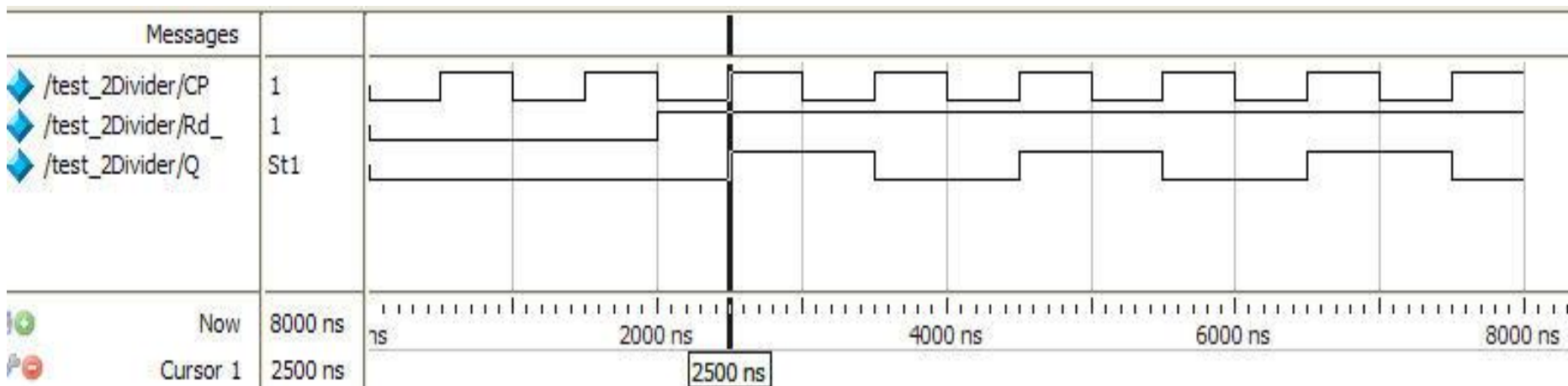


图 4.3.7 2 分频电路

(2) 激励块：给输入变量赋值。

```
`timescale 1 ns/ 1 ns
module test_2Divider();
    reg CP, Rd_;    wire Q;
    //调用设计块
    _2Divider U1 ( .CP(CP), .Q(Q), .Rd_(Rd_) );
    initial begin    //产生复位信号Rd_
        Rd_ = 1'b0;
        Rd_ = #2000 1'b1;
        #8000 $stop;
    end
    always begin    //产生时钟信号CP
        CP = 1'b0;
        CP = #500 1'b1;
        #500;
    end
end
endmodule
```

(3) 仿真波形 (用ModelSim)



由图可知，时钟CP的周期为1000ns，在2000ns之前，清零信号Rd_有效，输出Q被清零。在此之后，Rd_=1，在2500ns时，CP上升沿到来，Q=1；到下一个CP上升沿（3500ns）时，Q=0，再到下一个CP上升沿（4500ns）时，Q=1，.....，如此重复，直到8000ns时，系统任务\$stop被执行，仿真停止。

总之，在不考虑清零信号Rd_的作用时，每当CP上升沿到来时，触发器状态Q翻转一次。输出信号Q的频率正好是CP频率的二分之一，故称该电路为2分频电路。所谓分频电路，是指可将输入的高频信号变为低频信号输出的电路。

例5 试对图4.3.9所示电路进行建模，并给出仿真结果。

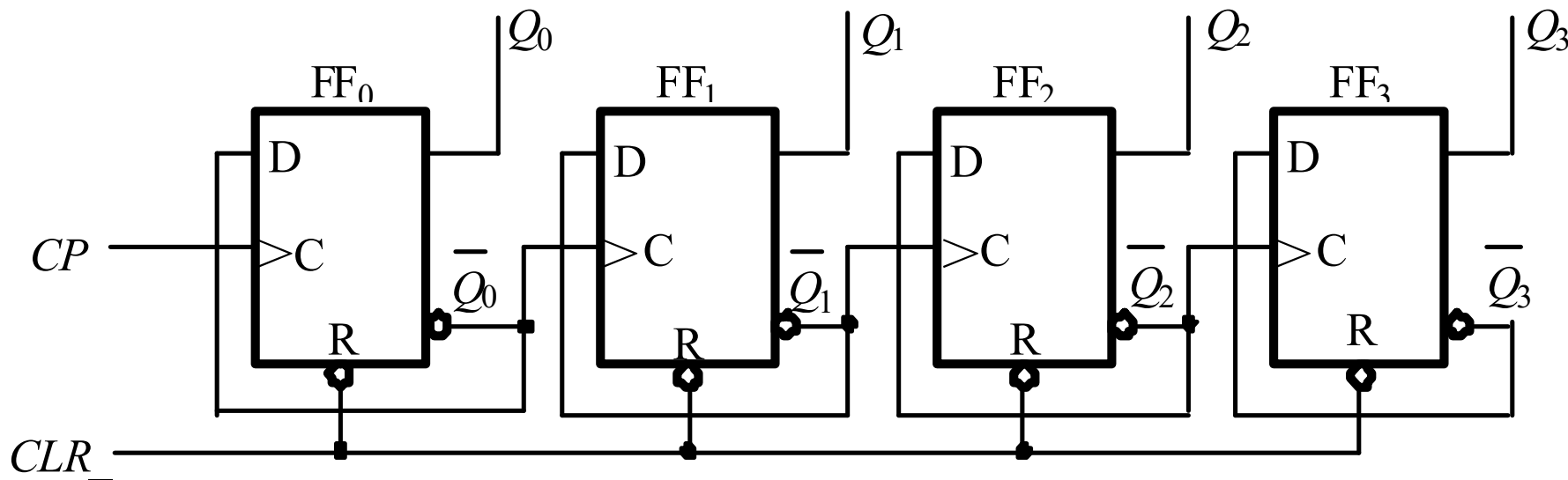


图4.3.9 4位异步二进制计数器逻辑图

解：（1）采用结构描述风格的代码如下。编写了两个模块，这两个模块可以放在一个文件中，文件名为Ripplecounter.v。

第一个主模块Ripplecounter作为设计的顶层，它实例引用分频器子模块_2Divider1共4次，第二个分频器子模块_2Divider1作为设计的底层。

```
/*==== 设计块: Ripplecounter.v =====*/  
module Ripplecounter (Q,CP,CLR_);  
    output [3:0]Q;  
    input CP, CLR_;  
  
    //实例引用分频器模块  
    _2Divider1 FF0 (Q[0],CP,CLR_);  
    //注意，引用时端口的排列顺序  
    _2Divider1 FF1 (Q[1],~Q[0],CLR_);  
    _2Divider1 FF2 (Q[2],~Q[1],CLR_);  
    _2Divider1 FF3 (Q[3],~Q[2],CLR_);  
endmodule
```

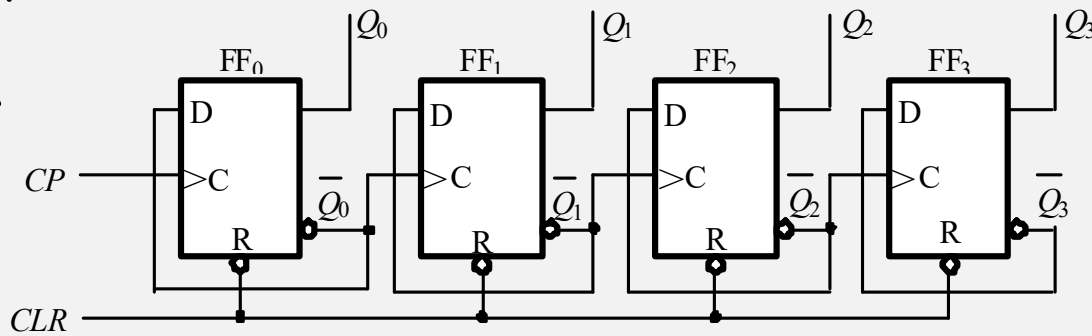


```
//分频器子模块
module _2Divider1 (Q,CP,Rd_);
    output reg Q;
    input CP,Rd_;
    always @(posedge CP or negedge Rd_)
        if(!Rd_)
            Q <= 1'b0;
        else
            Q <= ~Q;
endmodule
```



(2) 激励块：给输入变量 (CLR_和CP) 赋值。

```
/*==== 激励块: test_Ripplecounter.v ====*/  
module test_Ripplecounter();  
    reg CLR_, CP;  
    wire [3:0]Q;  
  
    Ripplecounter i1 (.CLR_(CLR_), .CP(CP), .Q(Q));  
  
    initial begin        // CLR_  
        CLR_ = 1'b0;  
        CLR_ = #20 1'b1;  
        #400 $stop;  
    end  
    always begin        // CP  
        CP = 1'b0;  
        CP = #10 1'b1;  
        #10;  
    end  
end  
endmodule
```

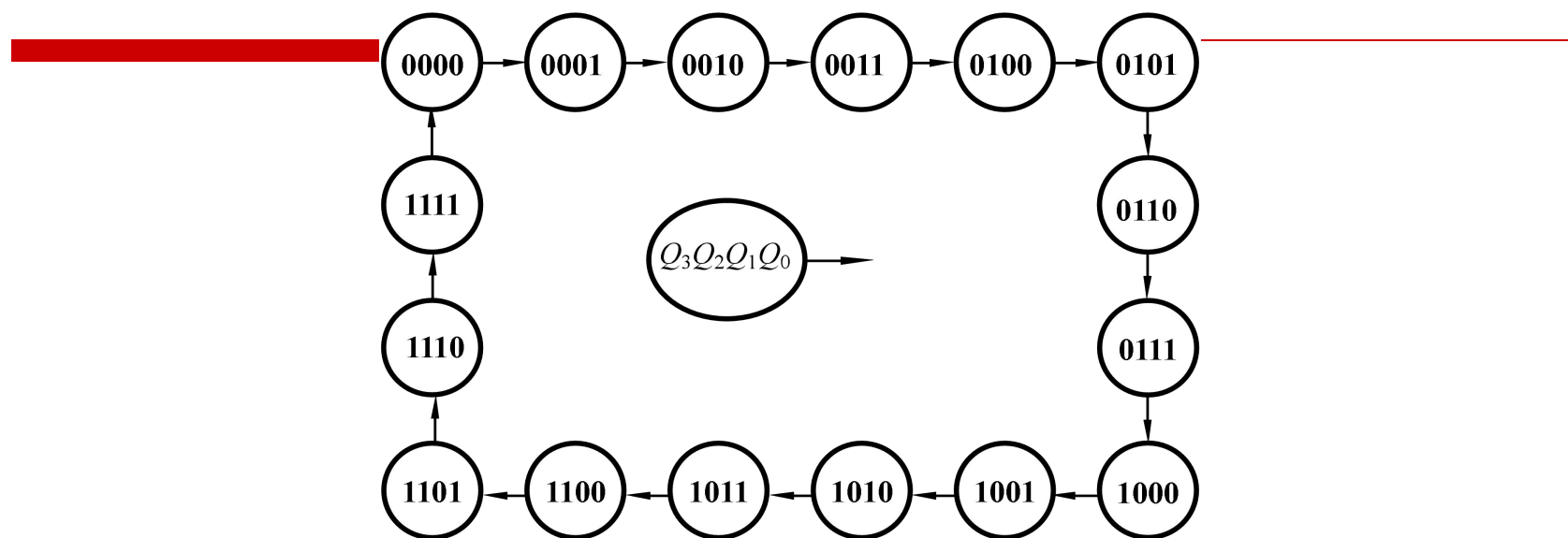


(3) 仿真波形如图4.3.10所示。



由图可知,

- 时钟CP的周期为20ns。
- 开始时, 清零信号CLR_有效 (0~20ns), 输出Q被清零。
- 20ns之后, CLR_一直为高电平,
- 在30ns时, CP上升沿到来, Q=0001; 到下一个CP上升沿 (50ns) 时, Q=0010,
- 再到下一个CP上升沿 (70ns) 时, Q=0011,, 如此重复, 到310ns时, Q=1111,
- 到330ns时, Q=0000,, 直到系统任务\$stop被执行, 仿真停止。



电路首先在 $CLR_$ 的作用下，输出被清零。此后当 $CLR_ = 1$ 时，每当 CP 上升沿到来时，电路状态 Q 就在原来二进制值的基础上增加1，即符合二进制递增计数的规律，直到计数值为1111时，再来一个 CP 上升沿，计数值回到0000，重新开始计数。故称该电路为4位二进制递增计数器。

可见，计数器实际上是对时钟脉冲进行计数，每到来一个时钟脉冲触发沿，计数器改变一次状态。

